MEDICATION SAFETY SIGNAL MONITOR · HAND-IN R08

AI reliability

Version 1.1, 5 November 2026: how often the label extraction is wrong, and how we know

The question this report answers

The brief asks: can you say how often the AI extraction is wrong, and how you know? Stretch: would you notice if it got worse?

One-line answer

On a small answer key (35 items: the Boxed Warning and Contraindications of 2 oxycodone labels, written by Claude and frozen before the run) it found 35 of 35 answer-key items and added 2 extra terms (precision at worst 35/37), so F1 = 0.97 against the proposed D015 target of 0.85 (met on this pilot key only; the benchmark comparison was not run). Separately, across all 9 sections it read, every one of its 448 quotes is found word for word in the label. No pharmacist has checked the key, so this is a pilot result, not proof.

What this is	What this is not
• How the AI label extraction works and where it sits in the pipeline • The two checks we run (grounding and a frozen answer key) and their results • What the results do not show, how we would notice a drop, and what the AI costs	• A pharmacist-validated accuracy figure: no pharmacist has reviewed the answer key yet • A score for the long label sections: they are not scored • A claim that the AI changes which signals are ranked: today it does not

How to read this. Chapter 1 is the one-page answer. Chapter 2 explains what the AI does. Chapter 3 gives the measurements, chapter 4 their limits. Chapter 5 is the monitoring, chapter 6 the cost, chapter 7 the next steps. Numbers from `numbers.json` are listed in the table at the end. Figures from other project records cite that record where they appear and are listed in *Figures quoted from project records*.

Contents	Page
01 · Summary in plain language	2
02 · What the AI does	4
03 · How we know how often it is wrong	5
04 · What this does not show	6
05 · How we would notice if it got worse	7
06 · What the AI costs	8
07 · What we would do next	9

01

Summary in plain language

A drug **label** (the official prescribing information, published on **DailyMed** as SPL XML, a structured format) lists the side effects the maker knows about. Our app flags side effects that people report to **FAERS** (the FDA's adverse event report database) but that the label does not mention. To know what the label mentions, an **LLM** (a large language model; here Claude Sonnet, called through **AIP**, Palantir Foundry's AI service) reads each safety section of the label and lists every reaction it names, each with an **exact quote** from the text. The pilot covers 13 drugs: 8 opioids (oxycodone, hydrocodone, morphine, fentanyl, hydromorphone, methadone, buprenorphine, tramadol) and 5 GLP-1 drugs (semaglutide, liraglutide, dulaglutide, tirzepatide, exenatide). **The AI has read only 2 of their labels, both for oxycodone.**

Each claim carries a label. Evidence labels say what kind of claim it is; status labels say how far the work has got:

Label	Meaning
OBSERVED	Read directly from a public source (FAERS, DailyMed, openFDA, RxNorm), with the query that produced it.
DERIVED	Computed by our pipeline from observed data, for example a PRR or a count per drug and reaction.
ASSUMED	Not verified: taken as true for now because the data or the brief is silent. It could be wrong.
DESIGNED	A design choice we made and wrote down (the decision log says why). It may be built; the status band says how far.
TESTED	Built, run, and checked against evidence.
PLANNED	On the plan; not built yet.
PROPOSED	Suggested; waiting for a decision.

Question	Answer	Evidence
Does it invent text?	No invented quotes found: 448 of 448 quotes appear word for word in the section (100%)	OBSERVED
Does it miss reactions?	On the scored sections it found 35 of 35 answer-key items (strict recall 1.00)	OBSERVED
Does it add wrong ones?	At most 2 extras: precision 35/37 (0.95), counting both extras as errors	OBSERVED
Overall	F1 0.97 on the 35-item key (2 short sections × 2 oxycodone labels) against the target of 0.85 (D015, proposed)	DERIVED
How sure are we?	Not very, yet: small answer key, written by Claude, long sections unscored	OBSERVED limits (ch. 4)
Would we notice a drop?	Grounding, errors and cache are checked every build; a model or prompt change forces re-extraction, then a manual re-score	TESTED

The short version

The AI's job today is **evidence**, not decisions. Every reaction it lists links to the exact quote and label version, which is the trust rule: every flag traces back to its source. Today that quote is stored in Foundry, not shown on the officer's screen (chapter 4).

The headline is honest only with its limits: it is a pilot score on the short sections of 2 labels, against an answer key that Claude wrote and froze before the run.

What this means for Dana (the safety officer)

The AI is not behind the flag she sees first. It read 0 semaglutide labels, so the hero signal (semaglutide × NAION) is *not in label* from the exact-text and synonym checks alone. The AI does not change her signal list today. NAION is a *known positive*, not a discovery: EMA PRAC (6 Jun 2025) and a WHO alert (27 Jun 2025) named it, and our 3 US labels do not mention it (DECISIONS learning 2026-11-05).

The AI's quote is not on her screen. It is stored with each labeled reaction and can be opened in Object Explorer (Foundry's object browser), not on her Workshop pages (report R04). A *not in label* signal has nothing to quote anyway.

Signals move only with a new FAERS quarter. FAERS is about 130 days behind (monitoring `faers_age_days`), and the FDA replaced FAERS with AEMS in March 2026, so the future feed is unverified (DQ20).

Only the project owner can submit a triage decision today, and monitoring alerts go to the owner, not to her. Both are production steps.

Gate: Summary answers the brief TESTED

Required evidence: A rate of wrong answers, how it was measured, and its limits.

Result: Met against the proposed D015 target (0.85) on the pilot key only: F1 0.97 on the 35-item key; the benchmark comparison D015 asks for (TAC 2017 / OnSIDES) was not run, so this is not independent. Separately, grounded 448/448 across all 9 sections; limits in chapter 4.

If it fails: Say "not measured" for the long sections rather than extend the number.

02

What the AI does

The extraction is one Foundry **transform** (Python code that reads datasets and writes new ones). Decision D041 records the design; approach A011 records how it was tested.

- **One call per safety section**, not per label: for example Boxed Warning, Contraindications, Warnings and Precautions, and Adverse Reactions.
- **Exact quote rule.** For every reaction the model must return the exact words from the section. Code then searches the section for that quote. A quote that is not there is marked `not_found`, so an invented quote cannot pass silently.
- **One reaction per item** (prompt p2). "Addiction, abuse, and misuse" becomes three items, because FAERS records one reaction term per row and matching needs single terms.
- **Extra fields.** An **assertion type** (what kind of statement it is: an adverse reaction seen in patients, a warned risk, a contraindicated condition, or other) and the population it applies to (for example neonates) are stored with each item.
- **Failures are visible.** A broken reply is `parse_error`, a failed call is `model_error`; both are kept and retried. A failure never counts as "the label does not mention it".
- **Cached.** The same text is never sent twice (see chapter 6 and report R06, entry O1).

Output dataset	One row per	Holds
`ai_label_section_extractions`	label section	status, raw model reply, tokens, time, `from_cache`, cache key
`ontology_labeled_reactions`	reaction in a section	reaction text, assertion type, population, **exact quote** and position, grounding result

Where it sits, and what it changes today

The AI reads the cleaned label sections and writes to the curated layer, before the signal step. The signal step checks each FAERS reaction against the label in three ways: (1) an AI term whose quote code found in the text, (2) the exact term in a safety section, (3) the reviewed synonym table (A012, D037 proposed, D043). The outcome is the same in any order; only the recorded match method (`label_match_method`) differs.

Said plainly

The AI does not change the signal list today. For the 2 pilot labels, plain text matching already finds every term the AI finds (A012). The AI's value now is the exact quote kept as evidence. Its value for matching depends on widening it to more labels, which waits for the pharmacist check.

Gate: The design is clear TESTED

Required evidence: Input, output, failure handling and the AI's effect on signals are written down.

Result: Met (D041 accepted, A011, A012).

If it fails: A reviewer cannot tell whether a flag rests on the AI or on text matching.

03

How we know how often it is wrong

Check	Catches	Misses	Result
Grounding: is each quote really in the section text?	Invented text (hallucination)	A real quote paired with the wrong item, and anything left out	448 / 448 exact OBSERVED
Answer key: does the list match a hand-made list?	Missed reactions (recall) and wrong extras (precision)	Only covers the sections in the key	F1 0.97 on 35 items DERIVED

The answer key

The answer key (file `eval/gold-set/step7-pilot-gold-v1.json`) lists every reaction, risk or contraindicated condition that the section text itself states, with accepted spellings. It covers the Boxed Warning and Contraindications sections of the 2 pilot labels: OxyContin (modern label format) and an older-format oxycodone/acetaminophen generic. It was **frozen before the model ran**, so it could not be tuned to the output.

Results

Measure	Prompt p1	Prompt p2 (in use)	Evidence
Recall, strict (term must equal an accepted spelling)	11 of 35 items (0.31)	**35 of 35 answer-key items** (1.00)	OBSERVED
Precision, upper bound (both extras counted wrong)	not scored	**35/37** (0.95)	OBSERVED
F1 (balance of recall and precision)	not scored	**0.97** (proposed target 0.85)	DERIVED
Grounding, all 9 sections (not only the key's)	all exact	**448 / 448**	OBSERVED

Why strict recall. FAERS reaction terms are MedDRA preferred terms (PT: the standard medical word list the FDA codes reports with). Matching works on single terms, so "addiction, abuse, and misuse" as one item would fail to match "abuse". Prompt p2 asks for one reaction per item; that is why the items found rose from 11 to 35 of 35 while grounding stayed complete.

The two extras. "Adverse drug effects" (too generic: a real false positive) and "liver transplant" (an outcome; arguably correct). Counting both as errors gives the precision upper bound.

How we know

Scoring is a script, not a judgement: `check_gold.py --strict` compares the run-2 terms with the frozen key. Grounding is a column on every output row; the monitoring check `ai.grounded_rate` reads it each build.

Gate: Error rate measured TESTED

Required evidence: Recall, precision and grounding from a key frozen before the run.

Result: Met on the pilot key: F1 0.97 (recomputed here from its parts: 0.97); not an independent benchmark.

If it fails: Re-score after any model, prompt or parser change before trusting the output.

04

What this does not show

These limits are part of the result. The F1 of 0.97 should never be quoted without them.

Three facts to say out loud

Said plainly: the AI read 0 labels for semaglutide, the drug behind our hero signal. The pilot is 2 oxycodone labels.

The AI does not change the signal list today. Exact text already finds every term it finds on those labels (A012).

Widening it is an owner decision (D041 v2, not yet taken). D041 says widen only after a pharmacist has checked the answer key.

Limit	Why it matters	What would fix it
Small key. Only the short sections of 2 labels; the answer key has 35 items.	One miss moves recall by about 3%. The number is evidence for the prompt change, not a stable rate.	A larger, stratified sample (chapter 7).
Claude wrote the key. Frozen first, but not reviewed by a pharmacist.	The same kind of model wrote the key and the answers; shared blind spots would not show.	A pharmacist labels the key, part of it blind.
Long sections are not scored. Warnings and Precautions and Adverse Reactions hold most of the 448 items.	Long sections are where lists are longest and misses are likeliest.	Score a sample of long-section items.
Grounding is not correctness. A quote can be real and the item still wrong (for example a mechanism, not a reaction).	448/448 says "no invented text", not "no errors".	Answer-key checks; a "term is in its quote" rate.
The AI does not change signals today.	So an AI error cannot yet move the ranked queue; this also means the AI's matching value is unproven.	Widen to more labels after the review.
The hero drug was not read by the AI. The AI read 0 semaglutide labels; the pilot is 2 oxycodone labels.	NAION is *not in label* from the exact-text and synonym checks only; the screen shows "Labels also read by AI: 0".	Widening the AI needs a D041 v2. That is an **owner decision**, not yet taken.
The quote is not on the officer's screen.	The trust evidence the AI adds is only in Object Explorer today (report R04).	A label-passage panel on page 1 (production step).
One model, one prompt. Claude Sonnet with prompt p2 only.	We cannot say another model would be better or worse.	Re-score other AIP models on the same key.

Gate: Limits stated TESTED

Required evidence: Each limit named, with its effect and the fix.

Result: Met. The pilot score is labelled as a pilot score throughout.

If it fails: The number reads as a validated accuracy, which it is not.

05

How we would notice if it got worse

The brief's stretch goal is to notice when the AI gets worse. We have three layers; the first two are built, the third is planned.

Layer	What it does	Status
1. Checks every build	The monitoring transform appends 21 checks per build to `monitoring_checks`. Three are about the AI: grounded rate (fail below 0.95), sections not parsed, and cache hit rate. First run: grounded 448/448, pass.	TESTED
2. Re-score on any change	The cache key includes the model, prompt and parser version. Changing any of them changes every key, so every section is extracted again (automatic). The new output is then scored against the answer key with `check_gold.py` before it is accepted (a manual script run today). Seen working when prompt p1 moved to p2 (A011 run 2).	TESTED (re-score manual)
3. Pharmacist-labelled sample	With another month: a pharmacist labels a stratified sample of long sections; it is scored the same way each release. Drift = F1 falling by more than 0.05 between releases.	PLANNED

Watch out

Drift today is a replay, not live. The pipeline drift table is rebuilt from dates the sources carry (`historical_replay`), because the pilot has not run for weeks. Live drift builds up only once builds run on a schedule, which is not set up yet.

No AI accuracy check runs every build yet. Grounding runs every build; accuracy against the answer key runs when the model, prompt or parser changes.

Gate: A drop would be noticed TESTED

Required evidence: AI checks run each build; a model or prompt change forces a re-score.

Result: Met for invented quotes and failures. Accuracy drift waits for the pharmacist sample.

If it fails: A silent model update could lower recall without any check firing.

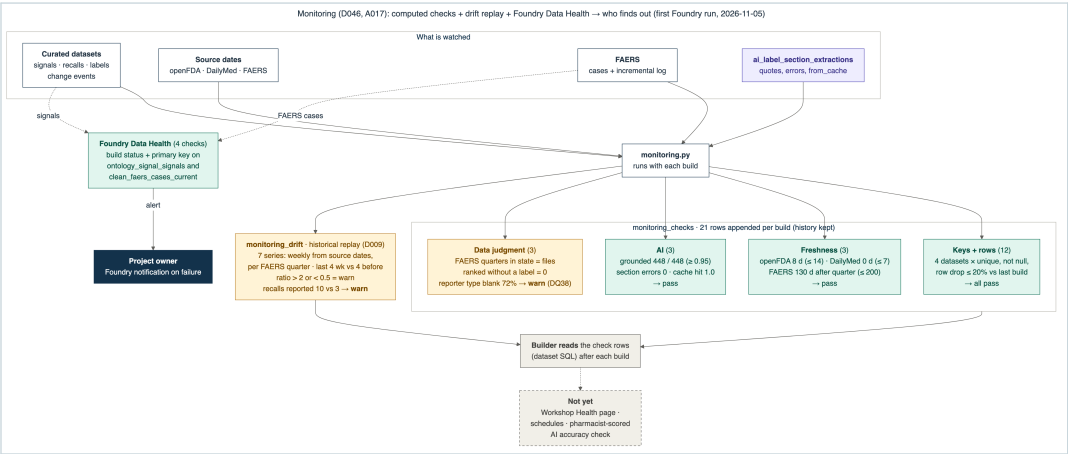


Figure 1. Monitoring: the checks per build (the AI group is in green, centre), drift replay and Foundry Data Health, and who finds out.
Source: docs/diagrams/png/35-handin-monitoring-v1.png (source: docs/diagrams/src/35-handin-monitoring-v1.mmd)

06

What the AI costs

Run	Model calls	Tokens	Evidence
Run 1 (prompt p1, 9 sections)	9	41,704 prompt + 29,072 completion = 70,776	OBSERVED
Rebuild with unchanged text	0	0 (9 of 9 sections from cache)	OBSERVED
Run 2 (prompt p2)	a fresh call per section	about 55% more output tokens than p1: more, shorter items (A011 run 2; learning 2026-11-04); exact count not in `numbers.json`	OBSERVED in A011

- **Dollar cost: not measured.** We recorded tokens, not the AIP price per token.
- **Cost when widened: not measured.** Extracting every label for all 13 drugs would be a one-time cost per label version, then near zero thanks to the cache ASSUMED. We have not estimated it with numbers.
- The cache key is the cost control: a label refresh with the same text costs nothing; only a new label version or a model or prompt change costs calls (report R06, O1).

Gate: Cost known TESTED

Required evidence: Tokens per run and calls on rebuild.

Result: Met for tokens and calls. Dollars and widened cost are not measured.

If it fails: Widening the AI could cost more than expected.

07

What we would do next

Step	Why	Status
A named pharmacist labels the answer key and a stratified sample of long-section items; Claude's draft is hidden on a blind part of the sample	Turns the pilot score into the pharmacist-labelled F1 per section that D015 (proposed) asks for; the blind part guards against simply agreeing with Claude	PROPOSED
Compare with OnSIDES (a public, CC BY 4.0 table of label-listed side effects)	An outside benchmark, not one we wrote	PROPOSED
Store and report a "term is in its quote" rate	Catches a real quote paired with the wrong term	PROPOSED
Score other AIP models on the same key	Model choice can be measured, not assumed	PROPOSED
Schedule builds; add an AI tile to a Health page	Live drift instead of replay	PLANNED

Take-away
We can say how often the AI is wrong on the part we scored, and how we know: two different checks, a key frozen before the run, and every number reproducible by a script. We cannot yet say it for the long sections, and we say so.

Figures quoted from project records (not in numbers.json)

Figure	Source
2 extra terms ("adverse drug effects", "liver transplant")	eval/ai-eval-v1.md
Recall and precision as decimals; one miss = about 3%; grounded share	DERIVED here from `numbers.json` fractions
0 semaglutide labels read by the AI; "Labels also read by AI: 0" on NAION	D041 pilot scope; Workshop page 1 (2026-11-05); rubric check
Grounded-rate check fails below 0.95	D046 (proposed); monitoring README; A017
Three AI checks among the per-build checks	foundry/pipelines/monitoring/README.md
Drift = F1 drop of more than 0.05 between releases	eval/ai-eval-v1.md (another month)
Prompt p2: about 55% more output tokens	A011 run 2; DECISIONS learning 2026-11-04
OnSIDES licence CC BY 4.0	docs/09-roadmap/another-month-sources-and-pharmacist-review.md

Numbers used in this report

Every number in the table below is looked up from `docs/11-handin/numbers.json` at build time. Source: Foundry SQL (repo SQL panel) after the baseline-gate rebuild of the whole downstream chain: raw_dailymed_spl → clean_labels → clean_label_product_map → ontology_objects → ai_label_extraction → ontology_signals → ontology_changes → monitoring. Build durations from the Foundry Builds page. Query: docs/11-handin/baseline-gate.md. Captured: 2026-11-05T07:50Z. Figures quoted from other project records (decision log, data-issues register, optimization log) name that record where they appear.

Key	Value
ai.d015_target_f1	0.85
ai.from_cache_on_rebuild	9
ai.gold_f1	0.97
ai.gold_precision_upper	35/37
ai.gold_recall_strict	35/35
ai.gold_recall_strict_p1	11/35
ai.grounded	448
ai.model_calls_on_rebuild	0
ai.pilot_labels	2
ai.reactions	448
ai.run1_tokens_completion	29,072
ai.run1_tokens_prompt	41,704
ai.sections	9
monitoring.checks_per_build	21
monitoring.faers_age_days	130
optimization.O1.after_tokens	0
optimization.O1.before_calls	9
scope.drugs_formulary	13
scope.glp1	5
scope.opioids	8